IDAT7215
Computer Programming for Product Development and Applications

Lecture 1-2: Python Installation and Python Fundamentals

Dr. Zulfiqar Ali

Python Installation

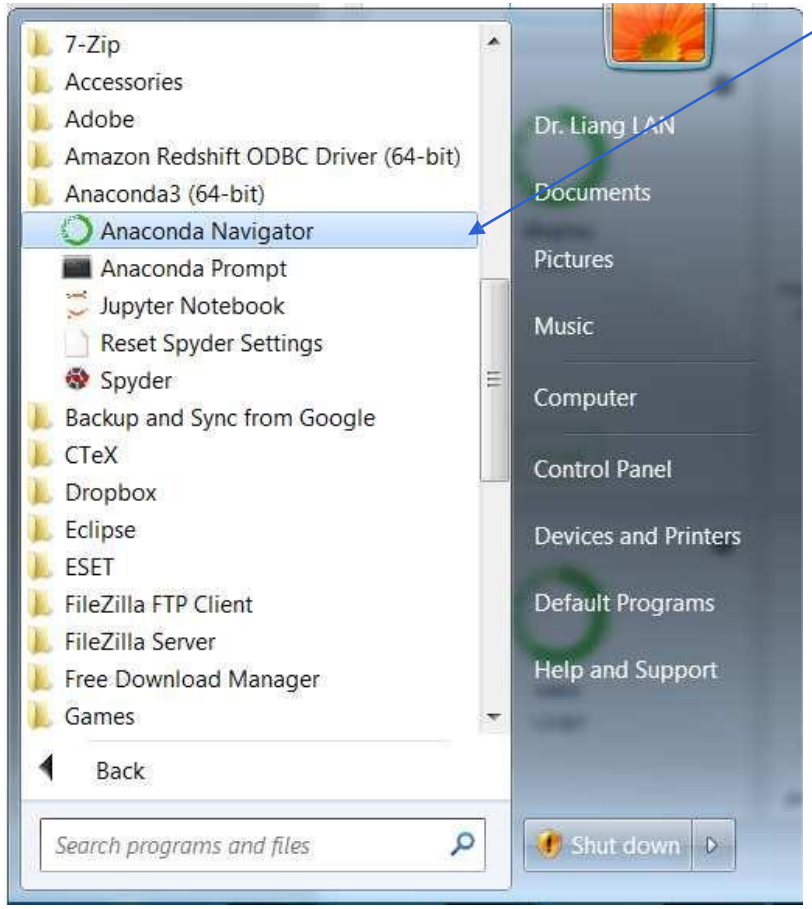
Python Installation

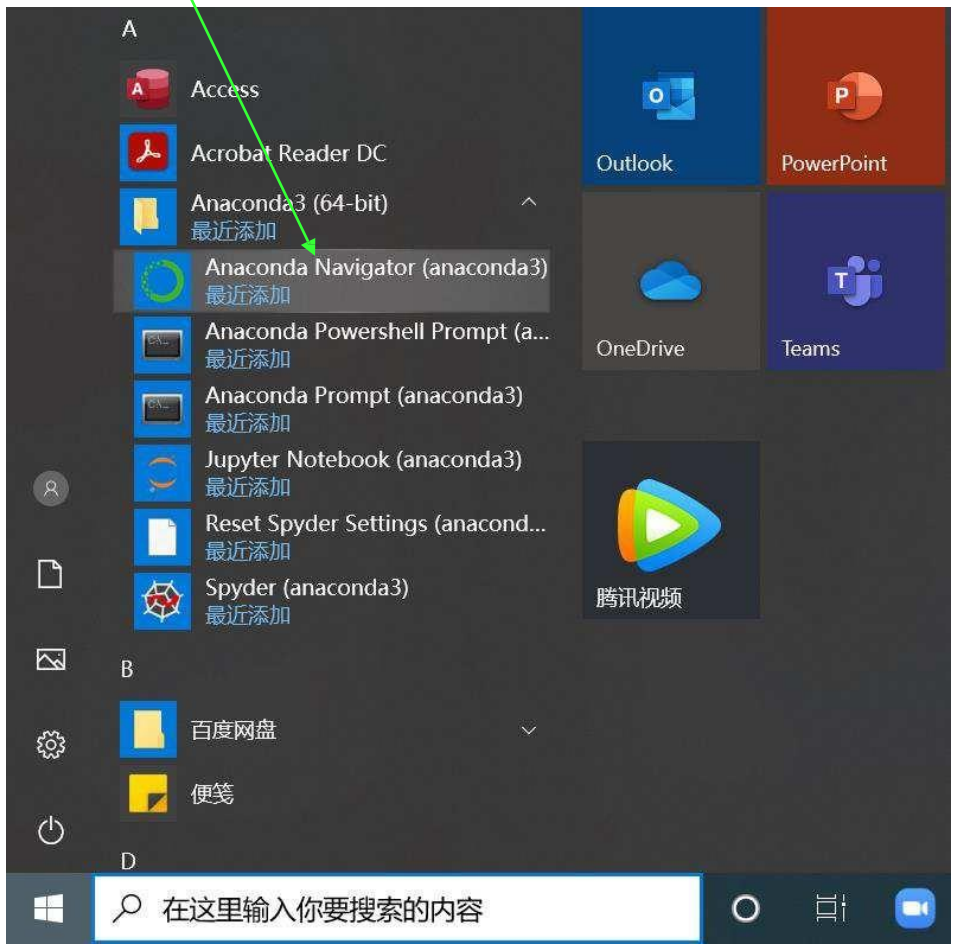
- Download and Install Anaconda
  - https://www.anaconda.com/products/individual
  - Choose your OS system, Recommend the 64-bit version
  - Install with the default configuration



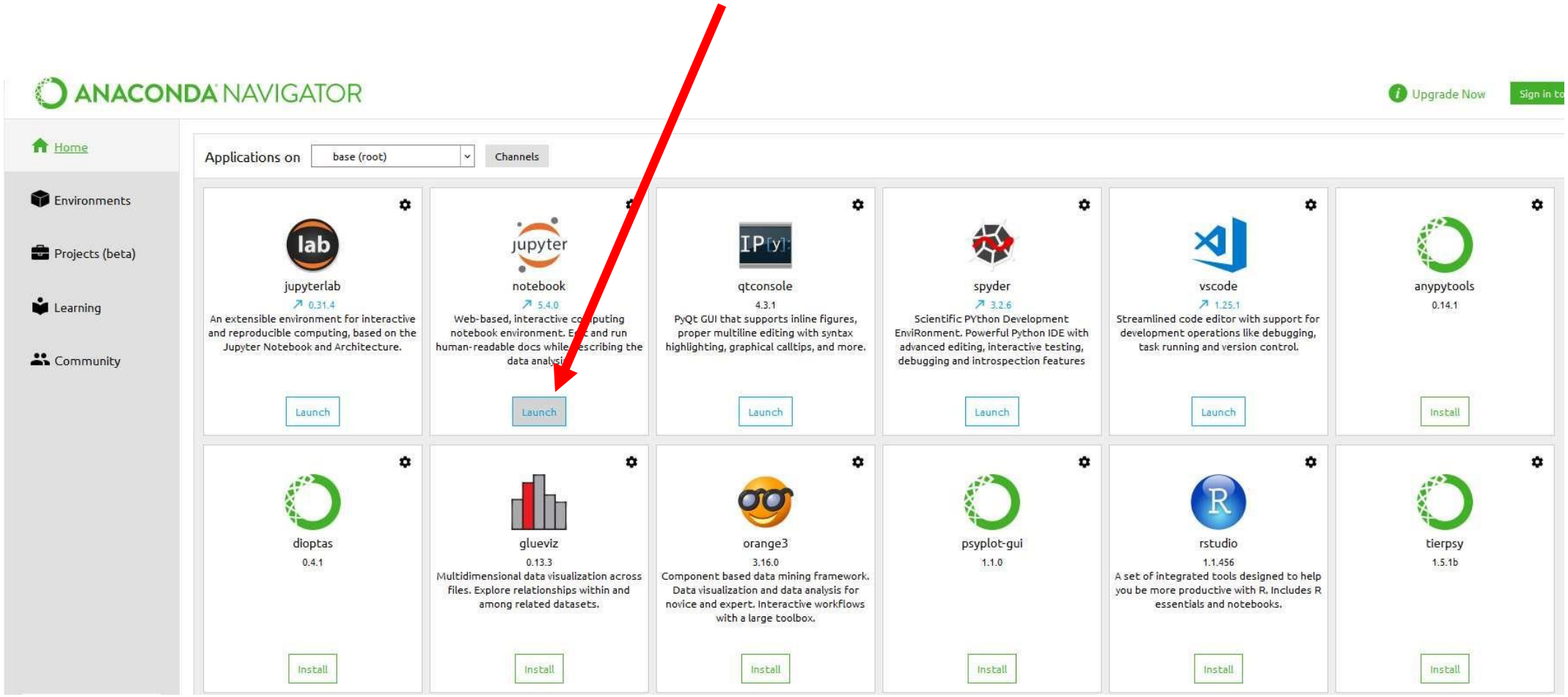
Launch Jupyter Notebook

- Open **Anaconda Navigator**.

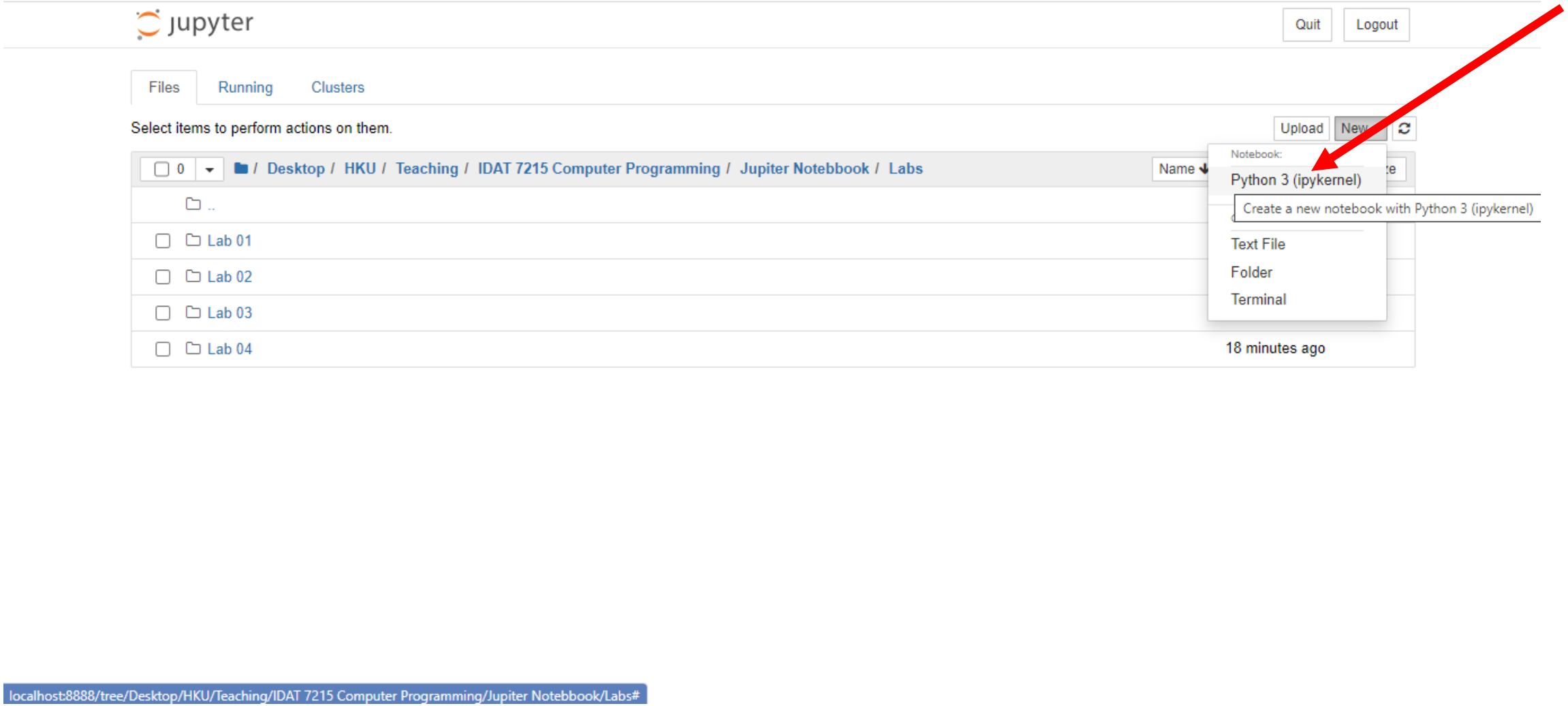




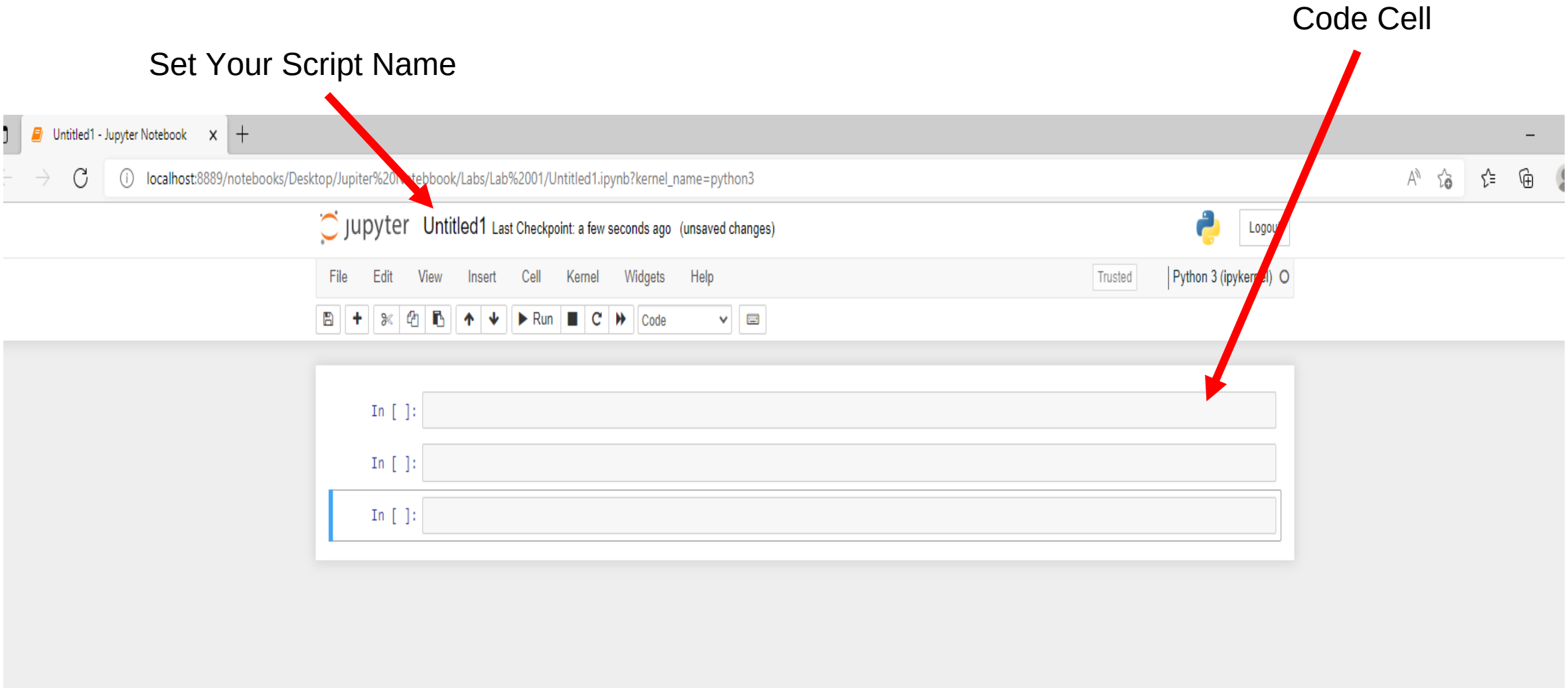
Launch Jupyter Notebook



Create your first notebook



Create your first notebook



Python fundamentals

Outline

- Variables and Types in Python
- Data Structures in Python
- Python Functions
- Control Flows

Variables and Types

- Variable
  - Specific, case-sensitive name
  - Call up value through variable name

- In Python, we do not have to explicitly declaim the type of the variable (e.g., ‘dog_name’) in any way.

- Python automatically detected the type of a variable.

```
In [2]: dog_name = 'Freddie'
        dog_age = 3
        dog_birth_year = 2006
        dog_weight = 30.2
        is_vaccinated = True

In [3]: dog_name
Out[3]: 'Freddie'
```

Rules of Variable Name: https://www.w3schools.com/python/python_variables.asp

Python Types

- The type of a variable can be queried by using the built-in function type.
- The type of a variable is not fixed.

```
In [3]: a = 3
        type(a)

Out[3]: int

In [4]: a = 'hello'
        type(a)

Out[4]: str
```

Basic data types in Python

- Basic data types
  - int (integers)
  - float (floating-point number)
  - str (string)
  - bool (boolean: True or False)
  - …
- The name of types act as conversion operators between types

```
In [2]: dog_name = 'Freddie'
        dog_age = 3
        dog_birth_year = 2006
        dog_weight = 30.2
        is_vaccinated = True
```

```
In [3]: dog_name
Out[3]: 'Freddie'
```

```
In [6]: print(int(2.8))
        print(float(2))
        print(int("123"))
        print(bool(-2), bool(2))
        print(str(234))
```

```
2
2.0
123
True True
234
```

Arithmetic Operations in Python

- Python supports the following basic arithmetic operations:

Operator	Operation	Example	
=	Assign a value to a variable	>>> a = 2	# a = 2
+	Addition	>>> a = 199 + 201	# a = 400
-	Subtraction	>>> a = 200 – 300	# a = -100
*	Multiplication	>>> a = 15.2 * 3	# a = 45.6
/	Division (Remark: it always returns a float)	>>> a = 15 / 2	# a = 7.5
//	Floor division (or integer division)	>>> a = 15 // 2	# a = 7
**	Power	>>> a = 4 ** 3	# a = 64
%	Remainder	>>> a = 15 % 2	# a = 1

String Operations in Python

- Create a string
  - A string is a sequence of characters.
  - The characters are specified either between single (‘) or double (“) quotes
  - E.g., `a = “abc”` or `a = ‘abc’`

```
# Create a string
message = "Hello World!"
print(message)
message = 'Hello World!'
print(message)
```

```
Hello World!
Hello World!
```

- Concatenate strings
  - `‘+’` operator
  - `join`

```
# concatenate string
message1 = 'Hello'
message2 = 'World!'
message = message1 + ' ' + message2
print(message)

message = (' ').join([message1, message2])
print(message)

message = (' ').join([message1, message2, message1, message2])
print(message)
```

```
Hello World!
Hello World!
Hello World! Hello World!
```

String Operations in Python

- Repeat a string

```
In [14]: message = "Hello World!"
         # Repeat string
         print(message*3) # repeat the message three times
```

```
Hello World!Hello World!Hello World!
```

- Tokenize a string

```
In [11]: # tokenize by space
         words = message.split(' ')
         print (words)
```

```
['Hello', 'World!']
```

- Replace

```
In [12]: # replace
         new_message = message.replace('Hello', 'Morning')
         print(new_message)
```

```
Morning World!
```

Data Structures In Python

- Lists
- Tuples
- Dictionaries
- Sets

Lists

- Probably the most fundamental data structure in Python.
- A list contains arbitrary number of elements that stored in sequential order.
- The elements are separated by commas and written between square brackets.
- Indexing begins from 0
- The elements don’t need to be of the same type.
- The elements of a list can be any type, including “List” itself.
- List is mutable.

List: Collection of Python Objects. Mutable

```
In [18]: # supports objects of different data types
         z = [1,4,'c',4, 2, 6]
         print (z)

[1, 4, 'c', 4, 2, 6]

In [19]: # Append to List
         z.append('a')
         print(z)

[1, 4, 'c', 4, 2, 6, 'a']

In [20]: z[0] = 'a'
         print(z)

['a', 4, 'c', 4, 2, 6, 'a']
```

Common Operations on List

- Get or set the i-th element in a list by indexing with square brackets

- Get or set elements in a list by slicing with square brackets
  - General format for slicing:
    `a[first:last:step]`



In [24]:
```
z = [1,4,'c',4, 2, 6]
# get the i-th element by indexing
print(z[2])
# set the i-th element by indexing
z[2] = 'a'
print(z)

# get elements by slicing
print(z[0:4])
print(z[0:4:2])
print(z[4:0:-2])

z[0:2] = ['a','a']
print(z)
```

```
c
[1, 4, 'a', 4, 2, 6]
[1, 4, 'a', 4]
[1, 'a']
[2, 'a']
['a', 'a', 'a', 4, 2, 6]
```

Common Operations on List

- Sorting sequences
  - `sort`: it modifies the original list
  - `sorted`: it returns a new sorted list and leaves the original list unchanged.
  - The parameter `reverse=True` (both to `sort` and `sorted`) can be given to get descending order of elements

In [28]:
```
# sorting a list
a = [3, 9, 4, 2]
a.sort()
print(a)

b = [3, 9, 4, 2]
print(sorted(b))
print(b)

print(sorted(b, reverse = True))
```

```
[2, 3, 4, 9]
[2, 3, 4, 9]
[3, 9, 4, 2]
[9, 4, 3, 2]
```

Common Operations on List

- Concatenate lists
  - `+`: return a new list. The original list is unchanged
  - `extend`: modify a list in place
  - `append`: add a single element to the end of a list
    - If append another list onto a list, it will treat another list as a single object

```
In [32]: # concatenate list
a = [1, 3, 2]
b = ['a', 'b', 'c']
a.extend(b)
print(a)

a = [1, 3, 2]
b = ['a', 'b', 'c']
print(a+b)
print(a)

a.append(0)
print(a)

a = [1, 3, 2]
a.append(b)
print(a)
```

```
[1, 3, 2, 'a', 'b', 'c']
[1, 3, 2, 'a', 'b', 'c']
[1, 3, 2]
[1, 3, 2, 0]
[1, 3, 2, ['a', 'b', 'c']]
```

Tuples

- Tuples are lists' **immutable** cousins.
- Pretty much anything you can do to a list that doesn't involve modifying it, you can do to a tuple.
- Tuples are defined by using parentheses (or nothing) instead of square brackets.

Tuple: Collection of Python Objects. Immutable

```
t =('a','b',3)
print(t)
print(type(t))
print(t[1])

t = 'a', 'b', 3
print(type(t))
```

```
('a', 'b', 3)
<class 'tuple'>
b
<class 'tuple'>
```

```
t[1]=2
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Input In [2], in <cell line: 1>()
----> 1 t[1]=2

TypeError: 'tuple' object does not support item assignment
```

Tuples

- Tuples are a convenient way to return multiple values from functions

```
def sum_and_product(x, y):
    sum_ = x + y
    product = x*y
    return sum_, product

sp = sum_and_product(10,5)
print(sp)

s, p = sum_and_product(10,5)
print(s)
print(p)
```

```
(15, 50)
15
50
```

Dictionaries

- Dictionary associates values with keys.
- Dictionary can be created by listing the comma separated key-value pairs in curly brackets. Keys and values are separated by a colon.
- Dictionary allows to quickly retrieve the value corresponding to a given key using square brackets.

```
In [47]: # dictionaries
         student_grades = {"Joel":80, "Tim": 95, "Anna": 100}
         print(student_grades["Joel"])

         80
```

Sets

- Set represents a collection of unique elements.
- Set can be created by listing its elements between curly brackets.
- Set can also be created by using `set()` itself

Sets collection of unique elements

In [48]:
```
# a set is not ordered
a = {1, 2, 3, 3, 3, 4, 5,'a'}
print(a)
a = set([1, 2, 3, 3, 3, 4, 5,'a'])
print (a)
a = set() # an empty set
a.add(1)
a.add(1)
print(a)
```

```
{1, 2, 3, 4, 5, 'a'}
{1, 2, 3, 4, 5, 'a'}
{1}
```

In [49]:
```
b = set('abaaaacdef')
print (b) # not ordered
```

```
{'d', 'b', 'c', 'f', 'a', 'e'}
```

Sets

- Sets are used for two main reasons:
  - Very fast membership checking (`in` is a very fast operation on sets
  - Find the unique items in a collections

In [78]:
```
# Membership checking
import time

word = ['a', 'an', 'at', 'b', 'bc', 'z']
%timeit 'b' in word


word = set(['a', 'an', 'at', 'b', 'bc', 'z'])
%timeit 'b' in word
```

```
44.9 ns ± 0.0664 ns per loop (mean ± std. dev.
25.3 ns ± 1.15 ns per loop (mean ± std. dev. of
```

- Sets are used less frequently than lists and dictionaries

In [79]:
```
# Unique items in a list
a = [1, 2, 1, 2, 1, 1, 3]
print(len(a))
unique_a = set(a)
print(unique_a)
print(len(unique_a))
```

```
7
{1, 2, 3}
3
```

Data Structures In Python

Type	Example	Description	Can we change the content within this collection?	Are the items in the collection ordered stored?
list	[1, 2, 3]	List: an ***mutable***, ***ordered*** collection	Yes – mutable	Yes – ordered
tuple	(1, 2, 3)	Tuple: an ***immutable, ordered*** collection	No – immutable	Yes – ordered
set	{1, 2, 3}	Set: ***Unordered*** collection of unique values	Yes – mutable	No – unordered
dict	{'a':1, 'b':2, 'c':3}	Dictionary: ***Unordered*** (key, value) mapping	Yes – mutable	No – unordered

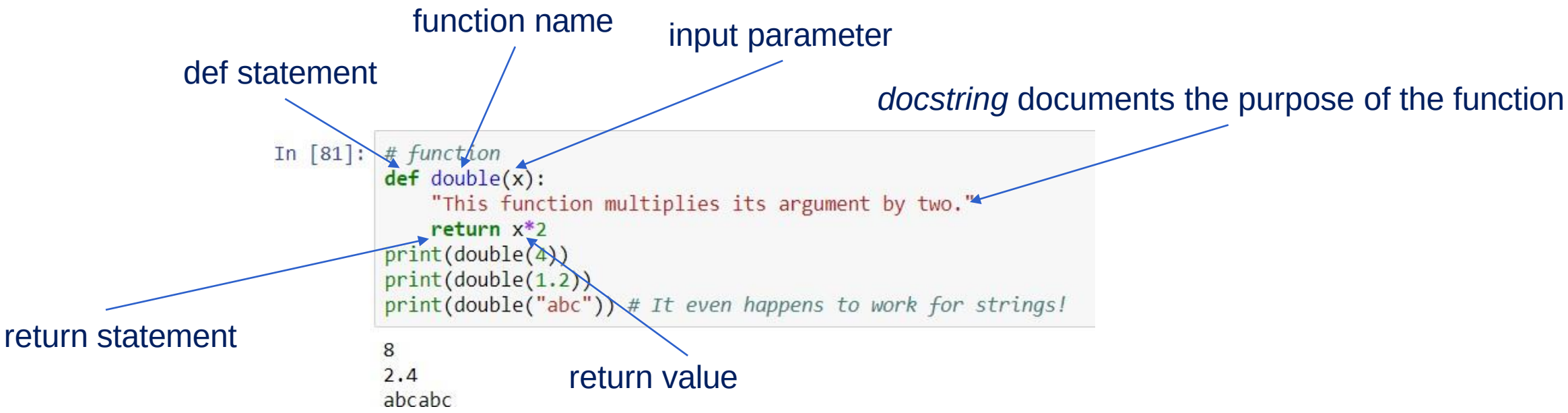
Python Functions

- A function is a rule for taking zero or more inputs and returning a corresponding out by performing a sequence of operations.
- When you define a function, you specify the name and the sequence of operations.
- Later, you can “call” the function by name.
- In Python, a function is defined with `def` statement.

In [81]:
```
# function
def double(x):
    "This function multiplies its argument by two."
    return x*2
print(double(4))
print(double(1.2))
print(double("abc")) # It even happens to work for strings!
```

```
8
2.4
abcabc
```

Python Functions



Python Functions

- Function parameter can also be given default values.

```
In [83]: def my_print(person = "Joe"):
             message = "Hello, " + person
             print(message)

         my_print("Peter")
         my_print()
```

```
Hello, Peter
Hello, Joe
```

Python Functions

- In addition to positional arguments we have seen so far, a function call can also have *named arguments*.

```
In [84]: def named(a, b, c):
             print("First:", a, "Second:", b, "Third:", c)

         named(5, 8, 7)
         named(5, c=7, b=8)
```

```
First: 5 Second: 8 Third: 7
First: 5 Second: 8 Third: 7
```

- Note that the named arguments didn't need to be in the same order as in the function definition. The named arguments must come after the positional arguments.

```
In [85]: named(a=5, 7, 8)
```

```
  File "<ipython-input-85-33e6d8dbc2a3>", line 1
    named(a=5, 7, 8)
                ^
SyntaxError: positional argument follows keyword argument
```

Control Flows in Python

- With control flow, you can execute certain code blocks conditionally and/or repeatedly: these basic building blocks can be combined to create sophisticated programs.

- Three different categories for control flows
  - Loops for repetitive tasks (`while, for`)
  - Decision making with the `if-else` statement
  - Exception-Handling (`break, continue, try-except`)

Loops for repetitive tasks

- In Python we have two kinds of loops: `while` and `for`

- While loop is used to execute a block of statements repeatedly until a given condition is satisfied.

In [87]:
```
# while loop
i=1
while i*i < 100:
    print("Square of", i, "is", i*i)
    i = i + 1
print("Finished printing all the squares below 100.")
```

```
Square of 1 is 1
Square of 2 is 4
Square of 3 is 9
Square of 4 is 16
Square of 5 is 25
Square of 6 is 36
Square of 7 is 49
Square of 8 is 64
Square of 9 is 81
Finished printing all the squares below 100.
```

Loops for repetitive tasks

- For loop are used for sequential traversal.

```
In [90]: # for loop
         for i in [1,1,2,3,4,5,6,7,8,9]:
             print("Square of", i, "is", i*i)
```

```
Square of 1 is 1
Square of 1 is 1
Square of 2 is 4
Square of 3 is 9
Square of 4 is 16
Square of 5 is 25
Square of 6 is 36
Square of 7 is 49
Square of 8 is 64
Square of 9 is 81
```

- At each iteration, the variable `i` refers to another value from the list in order.

Decision making with the if statement

- The `if-else` statement is used to decide whether a certain statement or block of statements will be executed or not.
  - if a certain condition is true then a block of statement is executed otherwise not.

In [91]:
```
# if-else
def absolute_value(x):
    if x >= 0:
        a=x
    else:
        a=-x
    return a

print(absolute_value(2))
print(absolute_value(-2))
```

```
2
2
```

General form of an if-else statements

```
if condition1:
    statement1_1
    statement1_2
    ...
elif condition2:
    statement2_1
    statement2_2
    ...
...
else:
    statementn_1
    statementn_2
    ...
```

In [1]:
```
c = -2
if c > 0:
    print("c is positive")
elif c<0:
    print("c is negative")
else:
    print("c is zero")
```

```
c is negative
```

Exception handling

- Sometimes, you will find that you will need to handle for exceptions.
- The exceptions could be invalid operations (e.g., divide by 0 or compute the square root of a negative number) or a class of values that you simply are not interested in.
- To handle these exceptions, you can use
  - `Break`
  - `Continue`
  - `Try-except`

Break

- `break` statement is only allowed inside a loop body.
- When `break` executes, the loop terminates.
- In practical use, a `break` statement is usually used with `if` statement to break a loop conditionally.

Break

- Breaking the loop, when the wanted element is found

In [2]:
```
# break
# find the first negative number in a list
l=[1,3,65,3,-1,56,-10]
for x in l:
    if x < 0:
        break
print("The first negative list element was", x)
```

```
The first negative list element was -1
```

Continue

- `Continue` statement is only allowed inside a loop body.
- When `continue` executes, the current iteration of the loop terminates, and the execution continues with next iteration of the loop.
- In practical use, a `continue` statement is usually used with if statement to executes conditionally.

Continue

- Stopping current iteration and continuing to the next one (do not break the loop).

In [5]:
```
# continue
# compute the square root of each element in a list
from math import sqrt
l=[1,3,65,3,-1,56,-10]
for x in l:
    if x < 0:
        continue
    print("Square root of " +str(x) + " is " + str(sqrt(x)))
```

```
Square root of 1 is 1.0
Square root of 3 is 1.7320508075688772
Square root of 65 is 8.06225774829855
Square root of 3 is 1.7320508075688772
Square root of 56 is 7.483314773547883
```

Try-except

- Unhandled exceptions will cause your program to crash. You can handle them using `try` and `except` statement
- If the code in the `try` block works, the `except` block is skipped.
- If the code in the `try` block fails, it jumps to the `except` section.

In [7]:
```
# try-exception example
try:
    print(0 / 0)
except ZeroDivisionError:
    print("cannot divide by zero")
```

```
cannot divide by zero
```

Without the try block, the program will crash and raise an error.

```
print(0/0)
```

```
---------------------------------------------
ZeroDivisionError                         Tra
<ipython-input-8-1babb3b33639> in <module>()
----> 1 print(0/0)

ZeroDivisionError: division by zero
```

Try-except

```
while True:
    x = int(input("Please enter a number: "))
    print("%s squared is %s" % (x, x**2))
```

```
Please enter a number: 2
2 squared is 4
Please enter a number: a
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-9-b935f19ed274> in <module>()
      1 while True:
----> 2     x = int(input("Please enter a number: "))
      3     print("%s squared is %s" % (x, x**2))

ValueError: invalid literal for int() with base 10: 'a'
```



```
while True:
    try:
        x = int(input("Please enter a number: "))
        print("%s squared is %s" % (x, x**2))
    except ValueError:
        print("Please enter a valid number!")
```

```
Please enter a number: 2
2 squared is 4
Please enter a number: a
Please enter a valid number!
Please enter a number: 3
3 squared is 9

Please enter a number:
```

https://www.w3schools.com/python/default.asp